

Coordenação Distribuída de Agentes Autônomos com Exclusão Mútua via Broker Pub/Sub

Isabelle da Silva Tobias
15525991
T04

Kevin Rodrigues Nunes
15676030
T94

Kauã Lima de Souza
15674702
T94

Victor Yodono
13829040
T94

Resumo— Este documento apresenta o planejamento e a arquitetura de um sistema distribuído de carrinhos autônomos que navegam em um grid $N \times N$ sem colisões. O problema central é a coordenação descentralizada de acesso a recursos compartilhados — as células do grid — sem a presença de um coordenador central. Para resolvê-lo, cada agente implementará um **algoritmo de exclusão mútua distribuída** (a definir), utilizando relógios lógicos para ordenação causal dos eventos e desempate de prioridade. Além da movimentação, o sistema utiliza um mecanismo de **eleição distribuída** para determinar qual carrinho atenderá cada novo pedido que surge no grid. A comunicação entre os agentes é realizada via um **broker pub/sub** (a definir), seguindo o padrão *publish/subscribe*. São detalhados os tópicos de comunicação, os componentes internos de cada agente, os fluxos de operação com e sem contenção de recursos, e o protocolo de eleição para atribuição de pedidos.

Nota: O broker pub/sub e o algoritmo de exclusão mútua ainda serão definidos. A tabela abaixo lista os candidatos em avaliação:

Decisão	Candidatos
Broker	MQTT (Mosquitto), RabbitMQ, Redis Pub/Sub, ZeroMQ
Exclusão Mútua	Ricart-Agrawala, Maekawa, Token Ring, Lamport
Relógio Lógico	Lamport, Vectors

Os exemplos a seguir utilizam MQTT e Ricart-Agrawala como ilustração, mas a arquitetura é independente dessas escolhas.

I. INTRODUÇÃO

O desenvolvimento de sistemas distribuídos apresenta desafios fundamentais relacionados à coordenação de processos concorrentes que competem por recursos compartilhados. Em particular, o problema da **exclusão mútua distribuída** — garantir que apenas um processo acesse um recurso crítico por vez, sem dependência de um coordenador centralizado — é um dos pilares clássicos da

área.

Este projeto propõe uma implementação prática desses conceitos através de um sistema multi-agente: carrinhos autônomos que navegam em um grid $N \times N$ e devem coordenar seus movimentos para evitar colisões. Cada célula do grid é tratada como um recurso compartilhado, e o acesso exclusivo a ela será garantido por um **algoritmo de exclusão mútua distribuída** (ainda a definir), com ordenação temporal fornecida por **relógios lógicos**.

A comunicação entre os agentes será mediada por um **broker pub/sub** (ainda a definir), seguindo o padrão *publish/subscribe*. O broker atuará como intermediário de mensagens, mas não participará da lógica de coordenação — toda a inteligência de decisão é distribuída entre os agentes.

Além da coordenação de movimento, o sistema incorpora um mecanismo de **eleição distribuída** para a atribuição de pedidos. Quando um novo pedido surge em uma célula do grid, os carrinhos disponíveis competem entre si através de um processo de eleição: cada candidato publica sua oferta (baseada em critérios como distância ao pedido e carga atual), e o vencedor — eleito de forma descentralizada — assume a responsabilidade de atendê-lo. Esse mecanismo elimina a necessidade de um despachante central e garante distribuição autônoma de tarefas entre os agentes.

II. FUNCIONAMENTO GERAL

O protocolo de movimentação de cada carrinho segue um fluxo baseado em requisição e concessão de acesso exclusivo à célula de destino:

- O carrinho determina a próxima célula $[x, y]$ para a qual deseja se mover.
- Publica um `lock_request` no tópico correspondente, incluindo seu *timestamp* lógico de Lamport.
- Os demais carrinhos avaliam a requisição: concedem acesso imediato se não disputam a mesma célula, ou

cedem prioridade ao menor *timestamp* (desempate pelo identificador do processo).

4. Ao receber aprovação de todos os demais agentes, o carrinho ocupa a célula e publica a liberação do lock.

III. TECNOLOGIAS

A Tabela I apresenta o *stack* tecnológico adotado no projeto, selecionado com foco em compatibilidade com os conceitos de sistemas distribuídos estudados na disciplina.

TABLE I: STACK TECNOLÓGICO DO SISTEMA DISTRIBUÍDO

Componente	Tecnologia
Comunicação	Broker Pub/Sub (a definir)
Linguagem	Python
Sincronização	Relógio Lógico (a definir)
Exclusão Mútua	Algoritmo distribuído (a definir)
Eleição Distribuída	Algoritmo baseado em métrica (distância/carga)

IV. ARQUITETURA DO SISTEMA DISTRIBUÍDO

A. Tópicos do Broker

A comunicação entre os agentes é estruturada hierarquicamente nos tópicos descritos na Tabela II. Essa organização permite que cada carrinho se inscreva seleti-

vamente nos tópicos relevantes, minimizando o tráfego desnecessário. Os nomes são ilustrativos (estilo MQTT) e serão adaptados ao broker escolhido.

TABLE II: HIERARQUIA DE TÓPICOS DO BROKER

Tópico	Descrição
carrinhos/{id}/posicao	Broadcast da posição atual
carrinhos/{id}/heartbeat	Sinal de vida (detecção de falhas)
grid/{x}/{y}/lock.request	Pedido de reserva de célula
grid/{x}/{y}/lock.response	Aprovação/negação
grid/{x}/{y}/lock.release	Liberação da célula
pedidos/novo	Novo pedido publicado no grid
pedidos/{id}/candidatura	Candidatura de um carrinho ao pedido
pedidos/{id}/eleito	Resultado da eleição (vencedor)

B. Componentes Internos de Cada Agente

Cada carrinho é modelado como um agente autônomo composto por cinco módulos que cooperam internamente, conforme ilustrado na Figura I.

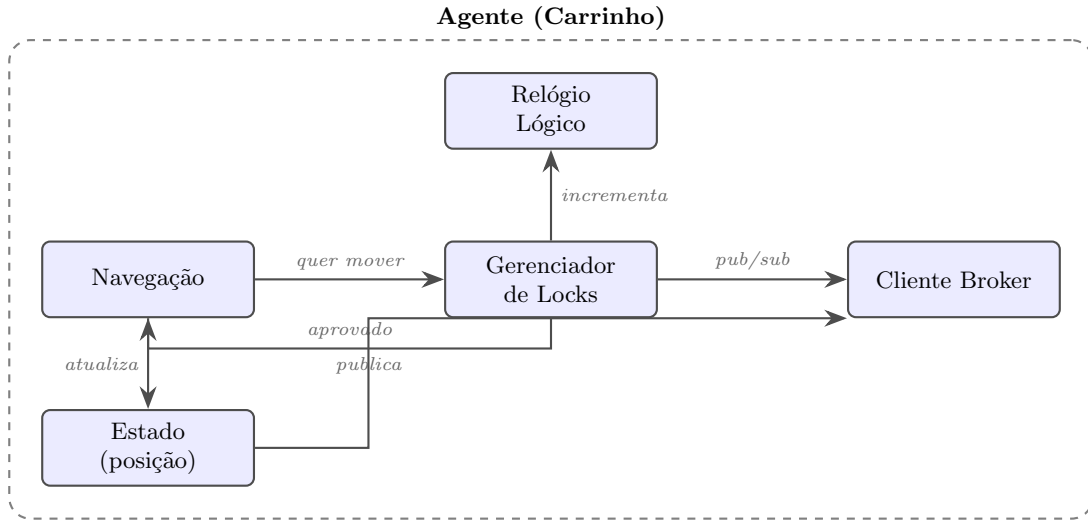


FIGURE I: DIAGRAMA DE COMPONENTES INTERNOS DE CADA AGENTE DISTRIBUÍDO

- **Navegação:** determina o próximo movimento e solicita o lock da célula de destino ao Gerenciador de Locks.
- **Gerenciador de Locks:** implementa o algoritmo de Ricart-Agrawala, gerenciando requisições, filas de espera e respostas.
- **Relógio de Lamport:** mantém a ordenação causal dos eventos, incrementando o contador a cada evento local e atualizando-o ao receber mensagens externas.

- **Cliente Broker:** realiza *publish* e *subscribe* nos tópicos do broker.
- **Estado (posição):** armazena e publica a posição atual do carrinho.

C. Fluxo Sem Contenção

A Figura II ilustra o cenário em que não há disputa pela mesma célula. O Carrinho A solicita acesso à célula [2, 3], recebe aprovação imediata e realiza o movimento.

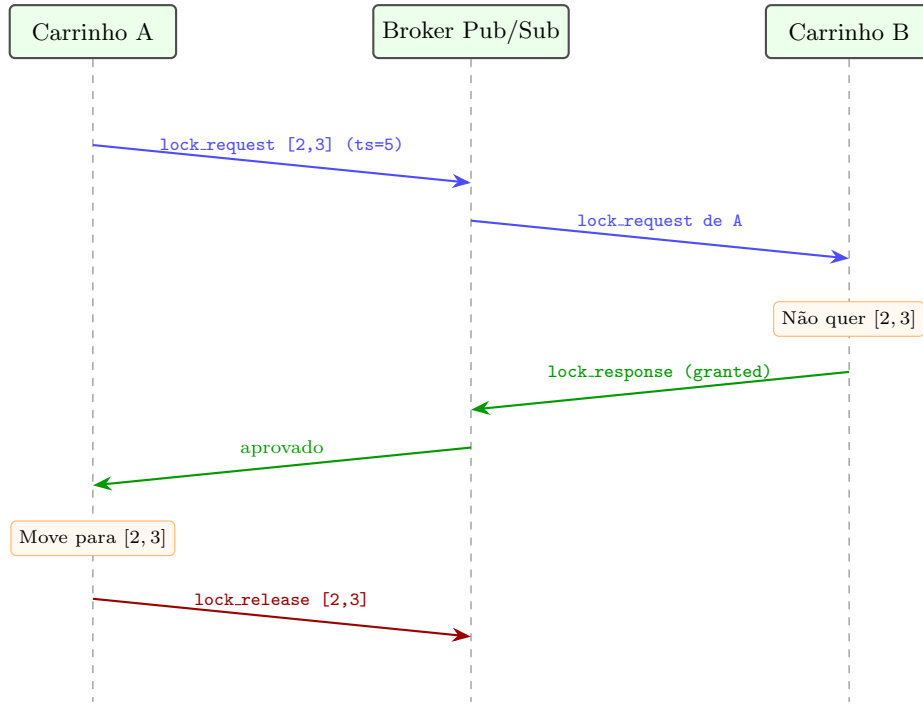


FIGURE II: DIAGRAMA DE SEQUÊNCIA — FLUXO SEM CONTENÇÃO DE RECURSOS

D. Fluxo Com Contenção (Mesma Célula)

Quando dois carrinhos disputam a mesma célula simultaneamente, o algoritmo de exclusão mútua resolve o conflito: o agente com menor *timestamp* lógico obtém pri-

oridade. Em caso de empate, o menor identificador de processo vence. O agente preterido enfileira a requisição e aguarda a liberação do recurso para tentar novamente. A Figura III ilustra esse cenário.

E. Eleição Distribuída para Atribuição de Pedidos

Além da coordenação de movimento, o sistema precisa decidir **qual carrinho atenderá cada pedido** que surge no grid. Essa decisão é feita de forma totalmente descentralizada, sem um despachante central, através de um processo de eleição distribuída.

O fluxo de eleição funciona da seguinte maneira:

1. Um novo pedido é publicado no tópico `pedidos/novo`, contendo a célula de origem $[x, y]$ e um identificador único.
2. Cada carrinho disponível calcula sua **métrica de aptidão** (por exemplo, distância Manhattan até o pedido, carga atual de tarefas, ou uma combinação ponderada de ambos).

3. Cada candidato publica sua métrica no tópico `pedidos/{id}/candidatura`, dentro de uma janela de tempo definida.
4. Após a janela de candidatura, cada carrinho compara as métricas recebidas. O agente com a **menor métrica** (mais apto) vence a eleição. Em caso de empate, o menor identificador de processo desempata.
5. O vencedor publica a confirmação no tópico `pedidos/{id}/eleito` e inicia a navegação até a célula do pedido.

Esse mecanismo garante que a atribuição de tarefas é **autônoma e distribuída**: nenhum nó central decide quem atende o quê. A Figura IV ilustra o fluxo de eleição entre três carrinhos.

F. Visão Geral

A Figura V apresenta a topologia geral do sistema. Múltiplos agentes (Carrinhos A, B, C, ...) comunicam-se exclusivamente através do Broker MQTT central, seguindo o padrão *publish/subscribe*. É importante ressaltar que o Broker não exerce papel de coordenação: ele apenas roteia mensagens. Toda a lógica de exclusão

mútua, ordenação causal, eleição de responsável por pedidos e tomada de decisão é inteiramente distribuída entre os agentes, em conformidade com os princípios fundamentais de sistemas distribuídos.

O sistema garante as seguintes propriedades:

- **Segurança (*safety*)**: no máximo um carrinho

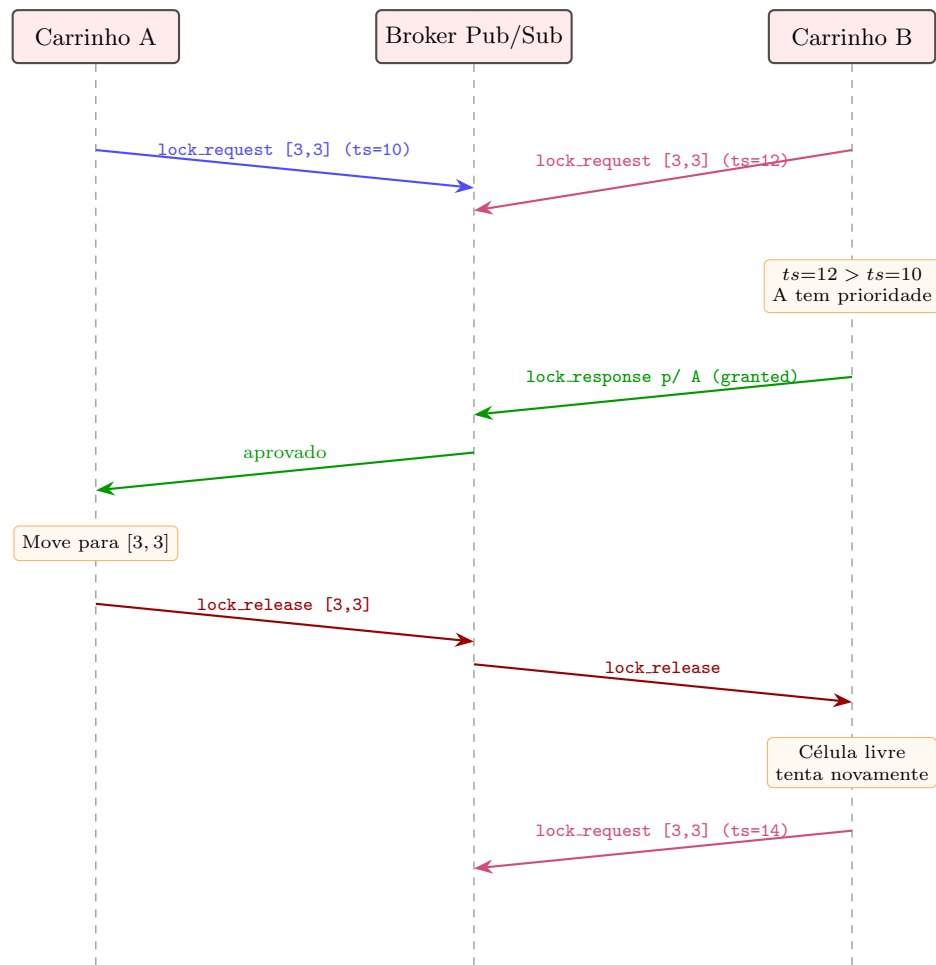


FIGURE III: DIAGRAMA DE SEQUÊNCIA — RESOLUÇÃO DE CONFLITO VIA EXCLUSÃO MÚTUA DISTRIBUÍDA

ocupa cada célula em qualquer instante.

- **Vivacidade (*liveness*):** todo carrinho que solicita acesso a uma célula eventualmente obtém permissão.
- **Ordenação causal:** os relógios de Lamport garantem uma ordenação total compatível com a causalidade dos eventos.
- **Distribuição autônoma de tarefas:** o mecanismo de eleição garante que pedidos são atribuídos ao agente mais apto sem intervenção central.
- **Deteção de falhas:** o mecanismo de *heartbeat* permite identificar agentes inativos e evitar *deadlocks*.

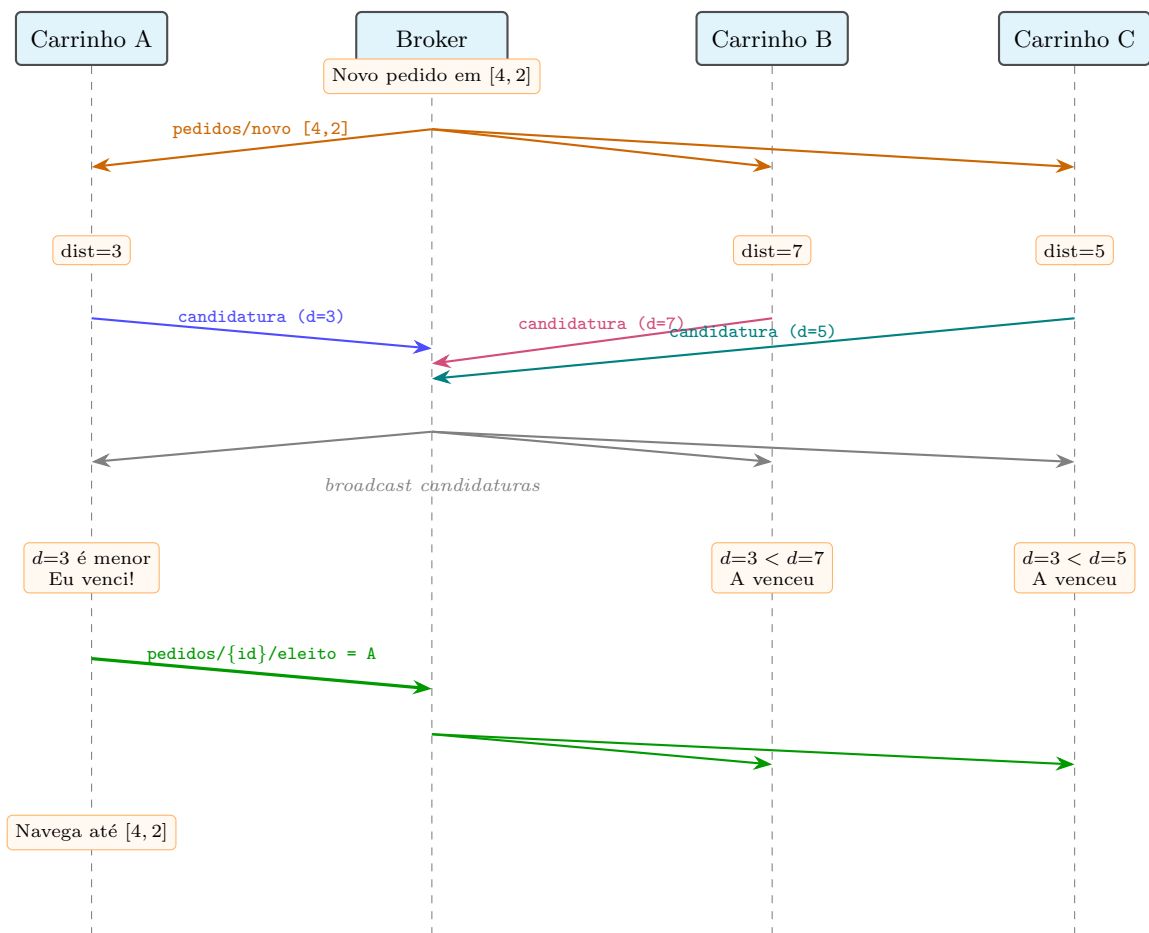


FIGURE IV: DIAGRAMA DE SEQUÊNCIA — ELEIÇÃO DISTRIBUÍDA PARA ATRIBUIÇÃO DE PEDIDO

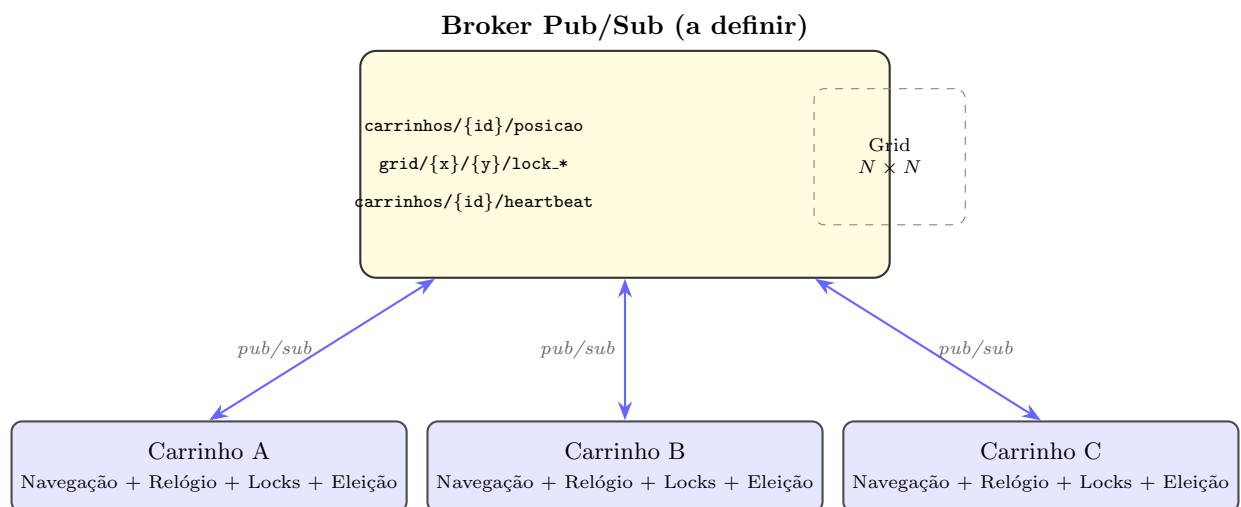


FIGURE V: VISÃO GERAL DA ARQUITETURA DISTRIBUÍDA DO SISTEMA MULTI-AGENTE